

Binary Search Templates

Binary search appears in two forms: searching an **array index**, or searching an **answer value**.

Master Quick Reference

#	Name	Description	Intuition	Time	Space	Key Implementation
704	Binary Search	Given a sorted array and a target, return its index. Return -1 if not found.	The target is one specific value; halve a closed range until you sit on it (or the range empties).	$O(\log n)$	$O(1)$	Standard
34	Find First and Last Position of Element in Sorted Array	Return <code>[first, last]</code> index of target. Return <code>{-1, -1}</code> if absent.	Find the boundary where "< target" flips to "≥ target" (first position), and the next boundary just past target.	$O(\log n)$	$O(1)$	Standard
35	Search Insert Position	Return the index where target is or would be inserted to keep the array sorted.	The insert position is the first index whose value is $\geq$ target — exactly <code>lowerBound</code> .	$O(\log n)$	$O(1)$	Standard
33	Search in Rotated Sorted Array	Array was sorted then rotated at an unknown pivot. Find target index, or -1.	Even rotated, one half is always sorted; check if target lies in that half and discard the other.	$O(\log n)$	$O(1)$	Standard
162	Find Peak Element	Return any index where <code>arr[k] > arr[k-1]</code> and <code>arr[k] > arr[k+1]</code> . Edges are $-\infty$.	Always walk uphill — an upward slope guarantees a peak to the right, so the slope direction is the "condition".	$O(\log n)$	$O(1)$	Standard
875	Koko Eating Bananas	Minimum eating speed so Koko finishes all piles in h hours (one pile per hour, at most <code>speed</code> bananas eaten).	Feasibility is monotonic in speed (faster always works), so binary search the speed axis for the smallest that finishes in time.	$O(n \log(\max(\text{piles})))$	$O(1)$	Standard
1011	Capacity to Ship Packages Within D Days	Minimum ship capacity to deliver all packages (in order) within d days.	Bigger capacity is always feasible, so binary search capacity for the smallest that ships within the day budget.	$O(n \log(\sum(\text{weights})))$	$O(1)$	Standard
410	Split Array Largest Sum	Split array into m non-empty subarrays to minimize the largest subarray sum.	A larger allowed max-sum needs fewer parts, so binary search that cap for the smallest splittable into $\leq m$ parts.	$O(n \log(\sum(\text{nums})))$	$O(1)$	Standard
1283	Find the Smallest Divisor Given a Threshold	Smallest positive divisor such that <code>sum of ceil(nums[i] / divisor) <= threshold</code> .	A bigger divisor shrinks the sum, so the condition flips false→true; binary search for the smallest divisor that fits the threshold.	$O(n \log(\max(\text{nums})))$	$O(1)$	Standard
1231	Divide Chocolate — MAXIMIZE (the one that differs)	Cut chocolate into k+1 pieces; keep the minimum-sweetness piece. Maximize that minimum.	A larger target-minimum yields fewer pieces, so the condition flips true→false; binary search for the largest minimum still giving $\geq k+1$ pieces.	$O(n \log(\sum(s)))$	$O(1)$	Standard
4	Median of Two Sorted Arrays	Find the median of two sorted arrays of sizes m and n in $O(\log(m+n))$ time.	Pick a split of the smaller array; the rest of the split is forced. Shrink the partition until the left half's max $\leq$ the right half's min, then the median falls out of the boundary values.	$O(\log(\min(m, n)))$	$O(1)$	search smaller array
69	Sqrt(x)	Compute the integer square root of a non-negative integer x without using <code>sqrt()</code> .	The condition <code>k*k <= x</code> holds true...true then flips false as k grows; find the largest k still true. Ceiling mid avoids the infinite loop at <code>i+1 == j</code> .	$O(\log x)$	$O(1)$	Standard
74	Search a 2D Matrix	Search for a target in an $m \times n$ matrix where each row is sorted and each row's first element > previous row's last.	The layout means the whole matrix reads as a single sorted sequence; map a flat index to <code>(row, col)</code> and run a standard closed binary search.	$O(\log(m \cdot n))$	$O(1)$	flatten index to 2D
81	Search in Rotated Sorted Array II	Search in a sorted, rotated array that may contain duplicates.	Duplicates can make both ends equal to the midpoint so neither half looks sorted; in that ambiguous case shrink both bounds by one and retry.	$O(\log n)$ average, $O(n)$ worst	$O(1)$	ambiguous duplicates
153	Find Minimum in Rotated Sorted Array	Find the minimum element in a sorted, rotated array with no duplicates.	The minimum is the single point where the order breaks; if <code>arr[k] > arr[j]</code> the break is to the right, otherwise the min is at k or left.	$O(\log n)$	$O(1)$	compare to right end
240	Search a 2D Matrix II	Search for a target in an $m \times n$ matrix where rows and columns are both sorted.	From the top-right, moving left strictly decreases and moving down strictly increases, so each comparison eliminates a full row or column.	$O(m + n)$	$O(1)$	start top-right corner
278	First Bad Version	Find the first bad version using a provided <code>isBadVersion(n)</code> API. Minimize API calls.	Versions are good...good then bad...bad once corrupted; this is the classic first-true boundary search.	$O(\log n)$	$O(1)$	Standard
374	Guess Number Higher or Lower	Guess the number between 1 and n using a provided <code>guess()</code> API. Minimize calls.	<code>guess(k)</code> tells you which half holds the answer (<code>-1</code> lower, <code>1</code> higher); halve the closed range until it returns 0.	$O(\log n)$	$O(1)$	Standard
475	Heaters	Given house and heater positions, find the minimum heater radius so every house is covered by some heater.	Each house needs the closest heater; the required radius is the largest of those closest distances. Find each house's nearest heater by binary search.	$O((m + n) \log m)$	$O(1)$	also check left neighbor
540	Single Element in a Sorted Array	Find the single non-duplicate element in a sorted array where all others appear twice. $O(\log n)$.	Before the unique element each pair starts at an even index; after it, the parity shifts. Force the midpoint even and check whether its partner matches to pick a half.	$O(\log n)$	$O(1)$	force k even
658	Find K Closest Elements	Find k integers in a sorted array closest to x, ordered by closeness then value.	The answer is a contiguous window of size k; search for its start by comparing the distances of the two window edges to x and sliding toward the closer side.	$O(\log(n - k) + k)$	$O(k)$	search window start

#	Name	Description	Intuition	Time	Space	Key Implementation
719	Find K-th Smallest Pair Distance	Find the k-th smallest absolute distance among all pairs in the array.	The count of pairs with distance $\leq d$ is monotonic in d , so binary search the distance axis; count pairs $\leq d$ with a sliding window over the sorted array.	$O(n \log n + n \log(\max \text{Dist}))$	$O(1)$	search distance value
852	Peak Index in a Mountain Array	Find the peak index in a mountain array (element greater than both neighbors).	An upward slope means the peak is strictly to the right; a downward slope means the peak is here or to the left.	$O(\log n)$	$O(1)$	Standard
1060	Missing Element in Sorted Array	Find the kth missing number in a sorted array.	Missing count at each index increases monotonically, so binary search for the first index whose missing count is $\geq k$, then offset from the previous element.	$O(\log n)$	$O(1)$	kth missing beyond array end
1428	Leftmost Column with at Least a One	Find the leftmost column with at least one '1' in a binary matrix (rows sorted, accessed via BinaryMatrix API).	Each row is sorted 0...0 1...1; starting top-right, move left on a 1 (column might be even further left) and down on a 0, tracking the leftmost 1 seen.	$O(\text{rows} + \text{cols})$	$O(1)$	start top-right corner
1482	Minimum Number of Days to Make m Bouquets	Find the minimum number of days to make m bouquets of k adjacent flowers.	Waiting more days only adds bloomed flowers, so feasibility is monotonic; binary search the day axis and greedily count adjacent runs of bloomed flowers.	$O(n \log(\max(\text{bloomDay})))$	$O(1)$	impossible if not enough flowers
1539	Kth Missing Positive Number	Find the kth missing positive integer.	At each index the number of missing positives so far is $\text{arr}[k] - (k+1)$; binary search the first index whose missing count is $\geq k$, then offset.	$O(\log n)$	$O(1)$	i missing-free slots + k
1818	Minimum Absolute Sum Difference	Find the minimum absolute sum difference after one substitution.	Total cost is the sum of absolute differences; one swap can only help where the closest available nums1 value beats the current difference. Find that closest value by binary search and track the best gain.	$O(n \log n)$	$O(n)$	also check left neighbor
1891	Cutting Ribbons	Find the maximum ribbon length such that m ribbons of that length can be cut.	Longer pieces produce fewer ribbons, so the count flips true→false as length grows; find the largest length still giving enough ribbons. Ceiling mid avoids the infinite loop.	$O(n \log(\max(\text{ribbons})))$	$O(1)$	closed $[i, j]$, result tracked

* #162 uses `j = n-1` (not `n`) — loop reads `arr[k+1]`, out of bounds if `j = n`.

All Four Templates

```
// 1. CLOSED [i, j] – exact match, return on hit
// MENTAL MODEL: target is a specific value sitting at some index; shrink a fully-closed range until you land on it.
// WHEN: "find this exact value in a sorted array"
int i = 0, j = n - 1;
while (i <= j) {
    int k = i + (j - i) / 2;
    if (arr[k] == target) {
        return k;
    } else if (arr[k] < target) {
        i = k + 1;
    } else {
        j = k - 1;
    }
}
return -1;

// 2. HALF-OPEN [i, j) – boundary search, answer falls out of i
// MENTAL MODEL: the predicate is false...false TRUE...true; find the FIRST position that satisfies the condition.
// WHEN: "first/last position", "insert position", "first index where condition holds"
int i = 0, j = n;          // j = n (or n-1 for #162)
while (i < j) {
    int k = i + (j - i) / 2;
    if (arr[k] < target) {
        i = k + 1; // lowerBound: <
    } else {
        j = k;      // upperBound: swap < for <=
    }
}
return i;

// 3. ANSWER SPACE MINIMIZE – smallest k where condition(k) is true
// MENTAL MODEL: condition goes false→TRUE as k grows (bigger k = more feasible); find the smallest k that flips it true.
// WHEN: "minimum X such that feasible(X)" – min speed, min capacity, smallest divisor
int i = minPossible, j = maxPossible;
while (i < j) {
    int k = i + (j - i) / 2;          // floor mid
    if (condition(k)) {
        j = k;
    } else {
        i = k + 1;
    }
}
return i;

// 4. ANSWER SPACE MAXIMIZE – largest k where condition(k) is true
// MENTAL MODEL: condition goes TRUE→false as k grows (bigger k = less feasible); find the largest k still true.
// WHEN: "maximum X such that feasible(X)" – max min-piece, max guarantee
int i = minPossible, j = maxPossible;
while (i < j) {
    int k = i + (j - i + 1) / 2;      // ceiling mid ← avoids infinite loop
    if (condition(k)) {
        i = k;
    } else {
        j = k - 1;
    }
}
return i;
```

How to Choose

Signal	Template
Sorted array, find exact value	Closed — return on <code>arr[k] == target</code>
First/last position, insert position	Half-open <code>lowerBound</code> / <code>upperBound</code>
Rotated array or slope search	Closed — pick sorted half each step
"minimum X such that feasible(X)"	Answer space MINIMIZE — floor mid
"maximum X such that feasible(X)"	Answer space MAXIMIZE — ceiling mid

Part 1 — Binary Search on Array Index

Detail Table

#	Name	Description	Interval	On <code>arr[k]==target</code>	Shrink left	Shrink right	Return
704	Binary Search	Find index of target; -1 if absent	Closed	<code>return k</code>	<code>i = k+1</code>	<code>j = k-1</code>	<code>-1</code>
34	First & Last Position	First and last index of target	Half-open	—	<code>i = k+1</code>	<code>j = k</code>	<code>{lower, upper-1}</code>
35	Search Insert Position	Index where target is or would be inserted	Half-open	—	<code>i = k+1</code>	<code>j = k</code>	<code>i</code>
33	Search in Rotated Array	Find target after unknown-pivot rotation	Closed	<code>return k</code>	<code>i = k+1</code>	<code>j = k-1</code>	<code>-1</code>
162	Find Peak Element	Any local peak index	Half-open*	—	<code>i = k+1</code>	<code>j = k</code>	<code>i</code>

#704 Binary Search

Description: Given a sorted array and a target, return its index. Return -1 if not found.

Template: Closed `[i, j]` — return immediately on exact hit.

Intuition: The target is one specific value; halve a closed range until you sit on it (or the range empties).

```
class Solution {
    public int search(int[] arr, int target) {
        int i = 0, j = arr.length - 1;
        while (i <= j) {
            int k = i + (j - i) / 2;
            if (arr[k] == target) {
                return k;
            } else if (arr[k] < target) {
                i = k + 1;
            } else {
                j = k - 1;
            }
        }
        return -1;
    }
}
```

Time $O(\log n)$ | **Space** $O(1)$

#34 Find First and Last Position of Element in Sorted Array

Description: Return `[first, last]` index of target. Return `{-1, -1}` if absent.

Template: Half-open `[i, j)`. `lowerBound` vs `upperBound` differ by one char: `<` vs `<=`.

Intuition: Find the boundary where "`< target`" flips to "`≥ target`" (first position), and the next boundary just past target.

```

class Solution {
    public int[] searchRange(int[] arr, int target) {
        int i = lowerBound(arr, target);
        if (i == arr.length || arr[i] != target) return new int[]{-1, -1};
        return new int[]{i, upperBound(arr, target) - 1};
    }
    private int lowerBound(int[] arr, int target) {
        int i = 0, j = arr.length;
        while (i < j) {
            int k = i + (j - i) / 2;
            if (arr[k] < target) {
                i = k + 1; // must go right
            } else {
                j = k; // k might be answer
            }
        }
        return i;
    }
    private int upperBound(int[] arr, int target) {
        int i = 0, j = arr.length;
        while (i < j) {
            int k = i + (j - i) / 2;
            if (arr[k] <= target) {
                i = k + 1; // still <= target, go right
            } else {
                j = k;
            }
        }
        return i;
    }
}

```

Time $O(\log n)$ | **Space** $O(1)$

#35 Search Insert Position

Description: Return the index where target is or would be inserted to keep the array sorted.

Template: Half-open `[i, j)` — identical to `lowerBound` from #34.

Intuition: The insert position is the first index whose value is $\geq$ target — exactly `lowerBound`.

```

class Solution {
    public int searchInsert(int[] arr, int target) {
        int i = 0, j = arr.length;
        while (i < j) {
            int k = i + (j - i) / 2;
            if (arr[k] < target) {
                i = k + 1;
            } else {
                j = k;
            }
        }
        return i;
    }
}

```

Time $O(\log n)$ | **Space** $O(1)$

#33 Search in Rotated Sorted Array

Description: Array was sorted then rotated at an unknown pivot. Find target index, or -1.

Template: Closed `[i, j]`. One half is always sorted — identify which, then test if target lies in it.

Intuition: Even rotated, one half is always sorted; check if target lies in that half and discard the other.

```

class Solution {
    public int search(int[] arr, int target) {
        int i = 0, j = arr.length - 1;
        while (i <= j) {
            int k = i + (j - i) / 2;
            if (arr[k] == target) return k;
            if (arr[i] <= arr[k]) { // left half [i..k] is sorted
                if (arr[i] <= target && target < arr[k]) {
                    j = k - 1;
                } else {
                    i = k + 1;
                }
            } else { // right half [k..j] is sorted
                if (arr[k] < target && target <= arr[j]) {
                    i = k + 1;
                } else {
                    j = k - 1;
                }
            }
        }
        return -1;
    }
}

```

Time $O(\log n)$ | **Space** $O(1)$

`arr[i] <= arr[k]` (not `<`) handles the edge case where `i == k`.

#162 Find Peak Element

Description: Return any index where `arr[k] > arr[k-1]` and `arr[k] > arr[k+1]`. Edges are $-\infty$.

Template: Half-open `[i, j)` with `j = n-1`. Binary search on slope: upward $\rightarrow$ peak is right, downward $\rightarrow$ peak is here or left.

Intuition: Always walk uphill — an upward slope guarantees a peak to the right, so the slope direction is the "condition".

```

class Solution {
    public int findPeakElement(int[] arr) {
        int i = 0, j = arr.length - 1;
        while (i < j) {
            int k = i + (j - i) / 2;
            if (arr[k] < arr[k + 1]) {
                i = k + 1; // slope up  $\rightarrow$  go right
            } else {
                j = k; // slope down  $\rightarrow$  peak at k or left
            }
        }
        return i;
    }
}

```

Time $O(\log n)$ | **Space** $O(1)$

Part 2 — Binary Search on Answer Space

Search on the **answer value** itself, not an array index. All use half-open `[i, j)` with `while (i < j)`.

Detail Table

#	Name	Description	Variant	i init	j init	Mid	Condition true	Condition false	Helper checks
875	Koko Eating Bananas	Min speed to finish all piles in h hours	MINIMIZE	1	max(piles)	floor	j = k	i = k+1	hours <= h
1011	Ship Within D Days	Min capacity to ship all packages in d days	MINIMIZE	max(w)	sum(w)	floor	j = k	i = k+1	days <= d
410	Split Array Largest Sum	Min largest sum splitting array into m parts	MINIMIZE	max(nums)	sum(nums)	floor	j = k	i = k+1	parts <= m
1283	Smallest Divisor	Smallest divisor so ceil-div-sum ≤ threshold	MINIMIZE	1	max(nums)	floor	j = k	i = k+1	divSum <= threshold
1231	Divide Chocolate	Max min-sweetness cutting into k+1 pieces	MAXIMIZE	min(s)	sum(s)/(k+1)	ceiling	i = k	j = k-1	pieces >= k+1

875 / 1011 / 410 / 1283 → identical structure, differ only in search bounds and condition helper.

1231 → only one that flips (MAXIMIZE): ceiling mid, i = k on success.

#875 Koko Eating Bananas

Description: Minimum eating speed so Koko finishes all piles in h hours (one pile per hour, at most speed bananas eaten).

Search space: [1, max(piles)]

Intuition: Feasibility is monotonic in speed (faster always works), so binary search the speed axis for the smallest that finishes in time.

```
class Solution {
    public int minEatingSpeed(int[] piles, int h) {
        int i = 1, j = Arrays.stream(piles).max().getAsInt();
        while (i < j) {
            int k = i + (j - i) / 2;
            if (canFinish(piles, k, h)) {
                j = k;
            } else {
                i = k + 1;
            }
        }
        return i;
    }
    private boolean canFinish(int[] piles, int speed, int h) {
        int hours = 0;
        for (int p : piles) hours += (p + speed - 1) / speed;
        return hours <= h;
    }
}
```

Time O(n log(max(piles))) | **Space** O(1)

#1011 Capacity to Ship Packages Within D Days

Description: Minimum ship capacity to deliver all packages (in order) within d days.

Search space: [max(weights), sum(weights)] — must fit heaviest; worst case ships one per day.

Intuition: Bigger capacity is always feasible, so binary search capacity for the smallest that ships within the day budget.


```

class Solution {
    public int shipWithinDays(int[] weights, int days) {
        int i = Arrays.stream(weights).max().getAsInt();
        int j = Arrays.stream(weights).sum();
        while (i < j) {
            int k = i + (j - i) / 2;
            if (canShip(weights, k, days)) {
                j = k;
            } else {
                i = k + 1;
            }
        }
        return i;
    }
    private boolean canShip(int[] weights, int cap, int days) {
        int d = 1, load = 0;
        for (int w : weights) {
            if (load + w > cap) { d++; load = 0; }
            load += w;
        }
        return d <= days;
    }
}

```

Time $O(n \log(\text{sum}(\text{weights})))$ | **Space** $O(1)$

#410 Split Array Largest Sum

Description: Split array into m non-empty subarrays to minimize the largest subarray sum.

Search space: $[\max(\text{nums}), \text{sum}(\text{nums})]$ — identical bounds to #1011.

Intuition: A larger allowed max-sum needs fewer parts, so binary search that cap for the smallest splittable into $\leq m$ parts.

```

class Solution {
    public int splitArray(int[] nums, int m) {
        int i = Arrays.stream(nums).max().getAsInt();
        int j = Arrays.stream(nums).sum();
        while (i < j) {
            int k = i + (j - i) / 2;
            if (canSplit(nums, k, m)) {
                j = k;
            } else {
                i = k + 1;
            }
        }
        return i;
    }
    private boolean canSplit(int[] nums, int cap, int m) {
        int parts = 1, sum = 0;
        for (int n : nums) {
            if (sum + n > cap) { parts++; sum = 0; }
            sum += n;
        }
        return parts <= m;
    }
}

```

Time $O(n \log(\text{sum}(\text{nums})))$ | **Space** $O(1)$

#1283 Find the Smallest Divisor Given a Threshold

Description: Smallest positive divisor such that $\text{sum of } \text{ceil}(\text{nums}[i] / \text{divisor}) \leq \text{threshold}$.

Search space: $[1, \max(\text{nums})]$

Intuition: A bigger divisor shrinks the sum, so the condition flips false→true; binary search for the smallest divisor that fits the threshold.

```
class Solution {
    public int smallestDivisor(int[] nums, int threshold) {
        int i = 1, j = Arrays.stream(nums).max().getAsInt();
        while (i < j) {
            int k = i + (j - i) / 2;
            if (divSum(nums, k) <= threshold) {
                j = k;
            } else {
                i = k + 1;
            }
        }
        return i;
    }
    private int divSum(int[] nums, int d) {
        int sum = 0;
        for (int n : nums) sum += (n + d - 1) / d;
        return sum;
    }
}
```

Time $O(n \log(\max(\text{nums})))$ | **Space** $O(1)$

#1231 Divide Chocolate ← MAXIMIZE (the one that differs)

Description: Cut chocolate into $k+1$ pieces; keep the minimum-sweetness piece. Maximize that minimum.

Search space: $[\min(s), \text{sum}(s)/(k+1)]$

Intuition: A larger target-minimum yields fewer pieces, so the condition flips true→false; binary search for the largest minimum still giving $\geq k+1$ pieces.

Ceiling mid is required. Concrete trace: with floor mid, $i=4, j=5 \rightarrow \text{mid}=4 \rightarrow$ on success $i=4 \rightarrow$ infinite loop; ceiling mid $i+(j-i+1)/2 \rightarrow \text{mid}=5 \rightarrow i=5 \rightarrow$ exits.

```
class Solution {
    public int maximizeSweetness(int[] s, int k) {
        int i = Arrays.stream(s).min().getAsInt();
        int j = Arrays.stream(s).sum() / (k + 1);
        while (i < j) {
            int m = i + (j - i + 1) / 2; // ceiling: guarantees progress when j == i+1
            if (canDivide(s, k + 1, m)) {
                i = m;
            } else {
                j = m - 1;
            }
        }
        return i;
    }
    private boolean canDivide(int[] s, int pieces, int minSweet) {
        int count = 0, sum = 0;
        for (int x : s) {
            sum += x;
            if (sum >= minSweet) { count++; sum = 0; }
        }
        return count >= pieces;
    }
}
```

Time $O(n \log(\text{sum}(s)))$ | **Space** $O(1)$

MINIMIZE vs MAXIMIZE Side by Side

	MINIMIZE (875,1011,410,1283)	MAXIMIZE (1231)
Mid:	floor $i + (j-i) / 2$	ceiling $i + (j-i+1) / 2$
If condition true:	$j = k$	$i = k$
If condition false:	$i = k + 1$	$j = k - 1$
Condition checks:	$\leq$ budget (feasible)	$\geq$ count (enough pieces)

Why ceiling for MAXIMIZE: when $i+1 == j$:

- floor gives $k = i \rightarrow$ if true, $i = i \rightarrow$ **infinite loop**
- ceiling gives $k = j \rightarrow$ if true, $i = j \rightarrow$ loop ends ✓

Additional Reference Problems

#4 Median of Two Sorted Arrays

Description: Find the median of two sorted arrays of sizes m and n in $O(\log(m+n))$ time.

Template: Half-open $[i, j)$ — binary search the partition point in the smaller array.

Intuition: Pick a split of the smaller array; the rest of the split is forced. Shrink the partition until the left half's max $\leq$ the right half's min, then the median falls out of the boundary values.

```
class Solution {
    public double findMedianSortedArrays(int[] nums1, int[] nums2) {
        if (nums1.length > nums2.length) { // ← VARIATION: search smaller array
            return findMedianSortedArrays(nums2, nums1);
        }
        int m = nums1.length, n = nums2.length;
        int half = (m + n + 1) / 2;
        int i = 0, j = m;
        while (i <= j) { // ← VARIATION: closed [i, j] on partition count
            int k = i + (j - i) / 2; // cut in nums1
            int cut2 = half - k; // cut in nums2
            int left1 = (k == 0) ? Integer.MIN_VALUE : nums1[k - 1];
            int right1 = (k == m) ? Integer.MAX_VALUE : nums1[k];
            int left2 = (cut2 == 0) ? Integer.MIN_VALUE : nums2[cut2 - 1];
            int right2 = (cut2 == n) ? Integer.MAX_VALUE : nums2[cut2];
            if (left1 <= right2 && left2 <= right1) {
                if (((m + n) & 1) == 1) {
                    return Math.max(left1, left2);
                } else {
                    return (Math.max(left1, left2) + Math.min(right1, right2)) / 2.0;
                }
            } else if (left1 > right2) {
                j = k - 1;
            } else {
                i = k + 1;
            }
        }
        return 0.0;
    }
}
```

Time $O(\log(\min(m, n)))$ | **Space** $O(1)$

#69 Sqrt(x)

Description: Compute the integer square root of a non-negative integer x without using `sqrt()`.

Template: Answer space MAXIMIZE — largest k with $k*k \leq x$.

Intuition: The condition `k*k <= x` holds true...true then flips false as k grows; find the largest k still true. Ceiling mid avoids the infinite loop at `i+1 == j`.

```
class Solution {
    public int mySqrt(int x) {
        if (x < 2) {
            return x;
        }
        int i = 1, j = x;
        while (i < j) {
            int k = i + (j - i + 1) / 2;           // ceiling mid (MAXIMIZE)
            if ((long) k * k <= x) {
                i = k;
            } else {
                j = k - 1;
            }
        }
        return i;
    }
}
```

Time $O(\log x)$ | **Space** $O(1)$

#74 Search a 2D Matrix

Description: Search for a target in an $m \times n$ matrix where each row is sorted and each row's first element > previous row's last.

Template: Closed `[i, j]` — treat the matrix as one flat sorted array of length $m*n$.

Intuition: The layout means the whole matrix reads as a single sorted sequence; map a flat index to `(row, col)` and run a standard closed binary search.

```
class Solution {
    public boolean searchMatrix(int[][] matrix, int target) {
        int m = matrix.length, n = matrix[0].length;
        int i = 0, j = m * n - 1;
        while (i <= j) {
            int k = i + (j - i) / 2;
            int value = matrix[k / n][k % n];           // ← VARIATION: flatten index to 2D
            if (value == target) {
                return true;
            } else if (value < target) {
                i = k + 1;
            } else {
                j = k - 1;
            }
        }
        return false;
    }
}
```

Time $O(\log(m*n))$ | **Space** $O(1)$

#81 Search in Rotated Sorted Array II

Description: Search in a sorted, rotated array that may contain duplicates.

Template: Closed `[i, j]` — like #33, but duplicates force a linear shrink when `arr[i] == arr[k] == arr[j]`.

Intuition: Duplicates can make both ends equal to the midpoint so neither half looks sorted; in that ambiguous case shrink both bounds by one and retry.

```

class Solution {
    public boolean search(int[] arr, int target) {
        int i = 0, j = arr.length - 1;
        while (i <= j) {
            int k = i + (j - i) / 2;
            if (arr[k] == target) {
                return true;
            }
            if (arr[i] == arr[k] && arr[k] == arr[j]) {      // ← VARIATION: ambiguous duplicates
                i++;
                j--;
            } else if (arr[i] <= arr[k]) {                  // left half sorted
                if (arr[i] <= target && target < arr[k]) {
                    j = k - 1;
                } else {
                    i = k + 1;
                }
            } else {                                       // right half sorted
                if (arr[k] < target && target <= arr[j]) {
                    i = k + 1;
                } else {
                    j = k - 1;
                }
            }
        }
        return false;
    }
}

```

Time $O(\log n)$ average, $O(n)$ worst | **Space** $O(1)$

#153 Find Minimum in Rotated Sorted Array

Description: Find the minimum element in a sorted, rotated array with no duplicates.

Template: Half-open `[i, j]` on slope — compare midpoint to the right end to pick the unsorted half.

Intuition: The minimum is the single point where the order breaks; if `arr[k] > arr[j]` the break is to the right, otherwise the min is at `k` or left.

```

class Solution {
    public int findMin(int[] arr) {
        int i = 0, j = arr.length - 1;
        while (i < j) {
            int k = i + (j - i) / 2;
            if (arr[k] > arr[j]) {                          // ← VARIATION: compare to right end
                i = k + 1;
            } else {
                j = k;
            }
        }
        return arr[i];
    }
}

```

Time $O(\log n)$ | **Space** $O(1)$

#240 Search a 2D Matrix II

Description: Search for a target in an $m \times n$ matrix where rows and columns are both sorted.

Template: Staircase search from the top-right corner (not a halving binary search, but the canonical $O(m+n)$ solution).

Intuition: From the top-right, moving left strictly decreases and moving down strictly increases, so each comparison eliminates a full row or column.

```

class Solution {
    public boolean searchMatrix(int[][] matrix, int target) {
        int row = 0, col = matrix[0].length - 1; // ← VARIATION: start top-right corner
        while (row < matrix.length && col >= 0) {
            int value = matrix[row][col];
            if (value == target) {
                return true;
            } else if (value > target) {
                col--;
            } else {
                row++;
            }
        }
        return false;
    }
}

```

Time $O(m + n)$ | **Space** $O(1)$

#278 First Bad Version

Description: Find the first bad version using a provided `isBadVersion(n)` API. Minimize API calls.

Template: Half-open `[i, j]` — find the first index where `isBadVersion` flips `false`→`true`.

Intuition: Versions are good...good then bad...bad once corrupted; this is the classic first-true boundary search.

```

/* The isBadVersion API is defined in the parent class VersionControl.
   boolean isBadVersion(int version); */
public class Solution extends VersionControl {
    public int firstBadVersion(int n) {
        int i = 1, j = n;
        while (i < j) {
            int k = i + (j - i) / 2;
            if (isBadVersion(k)) {
                j = k;
            } else {
                i = k + 1;
            }
        }
        return i;
    }
}

```

Time $O(\log n)$ | **Space** $O(1)$

#374 Guess Number Higher or Lower

Description: Guess the number between 1 and n using a provided `guess()` API. Minimize calls.

Template: Closed `[i, j]` — return on the exact hit when `guess` returns 0.

Intuition: `guess(k)` tells you which half holds the answer (`-1` lower, `1` higher); halve the closed range until it returns 0.

```

/* The guess API is defined in the parent class GuessGame.
   int guess(int num); returns -1, 1, or 0. */
public class Solution extends GuessGame {
    public int guessNumber(int n) {
        int i = 1, j = n;
        while (i <= j) {
            int k = i + (j - i) / 2;
            int response = guess(k);
            if (response == 0) {
                return k;
            } else if (response < 0) {           // my guess is too high
                j = k - 1;
            } else {
                i = k + 1;
            }
        }
        return -1;
    }
}

```

Time $O(\log n)$ | **Space** $O(1)$

#475 Heaters

Description: Given house and heater positions, find the minimum heater radius so every house is covered by some heater.

Template: Closed `[i, j]` — for each house, binary search the nearest heater; the answer is the max over all houses of the closest distance.

Intuition: Each house needs the closest heater; the required radius is the largest of those closest distances. Find each house's nearest heater by binary search.

```

class Solution {
    public int findRadius(int[] houses, int[] heaters) {
        Arrays.sort(heaters);
        int result = 0;
        for (int house : houses) {
            int i = 0, j = heaters.length - 1;
            while (i < j) {                     // find first heater >= house
                int k = i + (j - i) / 2;
                if (heaters[k] < house) {
                    i = k + 1;
                } else {
                    j = k;
                }
            }
            int distance = Math.abs(heaters[i] - house);
            if (i > 0) {                         // ← VARIATION: also check left neighbor
                distance = Math.min(distance, house - heaters[i - 1]);
            }
            result = Math.max(result, distance);
        }
        return result;
    }
}

```

Time $O((m + n) \log m)$ | **Space** $O(1)$

#540 Single Element in a Sorted Array

Description: Find the single non-duplicate element in a sorted array where all others appear twice. $O(\log n)$.

Template: Half-open `[i, j]` on even indices — a pair starting at an even index is intact iff it lies before the single element.

Intuition: Before the unique element each pair starts at an even index; after it, the parity shifts. Force the midpoint even and check whether its partner matches to pick a half.

```

class Solution {
    public int singleNonDuplicate(int[] arr) {
        int i = 0, j = arr.length - 1;
        while (i < j) {
            int k = i + (j - i) / 2;
            if ((k & 1) == 1) { // ← VARIATION: force k even
                k--;
            }
            if (arr[k] == arr[k + 1]) { // pair intact → single is to the right
                i = k + 2;
            } else {
                j = k;
            }
        }
        return arr[i];
    }
}

```

Time $O(\log n)$ | **Space** $O(1)$

#658 Find K Closest Elements

Description: Find k integers in a sorted array closest to x , ordered by closeness then value.

Template: Half-open $[i, j)$ — binary search the left boundary of the size- k window.

Intuition: The answer is a contiguous window of size k ; search for its start by comparing the distances of the two window edges to x and sliding toward the closer side.

```

class Solution {
    public List<Integer> findClosestElements(int[] arr, int k, int x) {
        int i = 0, j = arr.length - k; // ← VARIATION: search window start
        while (i < j) {
            int mid = i + (j - i) / 2;
            if (x - arr[mid] > arr[mid + k] - x) { // right edge is closer → move right
                i = mid + 1;
            } else {
                j = mid;
            }
        }
        List<Integer> result = new ArrayList<>();
        for (int p = i; p < i + k; p++) {
            result.add(arr[p]);
        }
        return result;
    }
}

```

Time $O(\log(n - k) + k)$ | **Space** $O(k)$

#719 Find K-th Smallest Pair Distance

Description: Find the k -th smallest absolute distance among all pairs in the array.

Template: Answer space MINIMIZE — smallest distance d with at least k pairs $\leq d$.

Intuition: The count of pairs with distance $\leq d$ is monotonic in d , so binary search the distance axis; count pairs $\leq d$ with a sliding window over the sorted array.


```

class Solution {
    public int smallestDistancePair(int[] nums, int k) {
        Arrays.sort(nums);
        int i = 0, j = nums[nums.length - 1] - nums[0];    // ← VARIATION: search distance value
        while (i < j) {
            int mid = i + (j - i) / 2;
            if (countPairs(nums, mid) >= k) {
                j = mid;
            } else {
                i = mid + 1;
            }
        }
        return i;
    }
    private int countPairs(int[] nums, int maxDist) {
        int count = 0, left = 0;
        for (int right = 0; right < nums.length; right++) {
            while (nums[right] - nums[left] > maxDist) {
                left++;
            }
            count += right - left;
        }
        return count;
    }
}

```

Time $O(n \log n + n \log(\maxDist))$ | **Space** $O(1)$

#852 Peak Index in a Mountain Array

Description: Find the peak index in a mountain array (element greater than both neighbors).

Template: Half-open $[i, j]$ on slope — walk uphill toward the peak.

Intuition: An upward slope means the peak is strictly to the right; a downward slope means the peak is here or to the left.

```

class Solution {
    public int peakIndexInMountainArray(int[] arr) {
        int i = 0, j = arr.length - 1;
        while (i < j) {
            int k = i + (j - i) / 2;
            if (arr[k] < arr[k + 1]) {    // slope up → go right
                i = k + 1;
            } else {
                j = k;
            }
        }
        return i;
    }
}

```

Time $O(\log n)$ | **Space** $O(1)$

#1060 Missing Element in Sorted Array

Description: Find the kth missing number in a sorted array.

Template: Half-open $[i, j]$ — the count of missing numbers before index k is $arr[k] - arr[0] - k$, which is monotonic.

Intuition: Missing count at each index increases monotonically, so binary search for the first index whose missing count is $\geq k$, then offset from the previous element.

```

class Solution {
    public int missingElement(int[] nums, int k) {
        int n = nums.length;
        int i = 0, j = n - 1;
        while (i < j) { // first index with missing count >= k
            int mid = i + (j - i) / 2;
            if (missingCount(nums, mid) >= k) {
                j = mid;
            } else {
                i = mid + 1;
            }
        }
        if (missingCount(nums, i) < k) { // ← VARIATION: kth missing beyond array end
            return nums[n - 1] + (k - missingCount(nums, n - 1));
        }
        return nums[i - 1] + (k - missingCount(nums, i - 1));
    }
    private int missingCount(int[] nums, int index) {
        return nums[index] - nums[0] - index;
    }
}

```

Time $O(\log n)$ | **Space** $O(1)$

#1428 Leftmost Column with at Least a One

Description: Find the leftmost column with at least one '1' in a binary matrix (rows sorted, accessed via BinaryMatrix API).

Template: Staircase walk from the top-right corner across rows that are individually sorted.

Intuition: Each row is sorted 0...0 1...1; starting top-right, move left on a 1 (column might be even further left) and down on a 0, tracking the leftmost 1 seen.

```

/* interface BinaryMatrix {
    public int get(int row, int col);
    public List<Integer> dimensions();
} */
class Solution {
    public int leftMostColumnWithOne(BinaryMatrix binaryMatrix) {
        List<Integer> dim = binaryMatrix.dimensions();
        int rows = dim.get(0), cols = dim.get(1);
        int row = 0, col = cols - 1; // ← VARIATION: start top-right corner
        int result = -1;
        while (row < rows && col >= 0) {
            if (binaryMatrix.get(row, col) == 1) {
                result = col;
                col--;
            } else {
                row++;
            }
        }
        return result;
    }
}

```

Time $O(\text{rows} + \text{cols})$ | **Space** $O(1)$

#1482 Minimum Number of Days to Make m Bouquets

Description: Find the minimum number of days to make m bouquets of k adjacent flowers.

Template: Answer space MINIMIZE — smallest day count d that yields $\geq m$ bouquets.

Intuition: Waiting more days only adds bloomed flowers, so feasibility is monotonic; binary search the day axis and greedily count adjacent runs of bloomed flowers.

```

class Solution {
    public int minDays(int[] bloomDay, int m, int k) {
        long need = (long) m * k;
        if (need > bloomDay.length) { // ← VARIATION: impossible if not enough flowers
            return -1;
        }
        int i = Arrays.stream(bloomDay).min().getAsInt();
        int j = Arrays.stream(bloomDay).max().getAsInt();
        while (i < j) {
            int k2 = i + (j - i) / 2;
            if (canMake(bloomDay, m, k, k2)) {
                j = k2;
            } else {
                i = k2 + 1;
            }
        }
        return i;
    }
    private boolean canMake(int[] bloomDay, int m, int k, int day) {
        int bouquets = 0, adjacent = 0;
        for (int bloom : bloomDay) {
            if (bloom <= day) {
                adjacent++;
                if (adjacent == k) {
                    bouquets++;
                    adjacent = 0;
                }
            } else {
                adjacent = 0;
            }
        }
        return bouquets >= m;
    }
}

```

Time $O(n \log(\max(\text{bloomDay})))$ | **Space** $O(1)$

#1539 Kth Missing Positive Number

Description: Find the kth missing positive integer.

Template: Half-open $[i, j]$ — missing count before index k is $\text{arr}[k] - (k + 1)$, monotonic non-decreasing.

Intuition: At each index the number of missing positives so far is $\text{arr}[k] - (k+1)$; binary search the first index whose missing count is $\geq k$, then offset.

```

class Solution {
    public int findKthPositive(int[] arr, int k) {
        int i = 0, j = arr.length;
        while (i < j) { // first index with missing count >= k
            int mid = i + (j - i) / 2;
            if (arr[mid] - (mid + 1) >= k) {
                j = mid;
            } else {
                i = mid + 1;
            }
        }
        return i + k; // ← VARIATION: i missing-free slots + k
    }
}

```

Time $O(\log n)$ | **Space** $O(1)$

#1818 Minimum Absolute Sum Difference

Description: Find the minimum absolute sum difference after one substitution.

Template: Closed $[i, j]$ — for each element binary search a sorted copy of `nums1` for the value closest to `nums2[i]`.

Intuition: Total cost is the sum of absolute differences; one swap can only help where the closest available `nums1` value beats the current difference. Find that closest value by binary search and track the best gain.

```
class Solution {
    public int minAbsoluteSumDiff(int[] nums1, int[] nums2) {
        int MOD = 1_000_000_007;
        int n = nums1.length;
        int[] sorted = nums1.clone();
        Arrays.sort(sorted);
        long total = 0;
        int maxGain = 0;
        for (int p = 0; p < n; p++) {
            int diff = Math.abs(nums1[p] - nums2[p]);
            total += diff;
            int target = nums2[p];
            int i = 0, j = sorted.length - 1;
            while (i < j) { // first value >= target
                int k = i + (j - i) / 2;
                if (sorted[k] < target) {
                    i = k + 1;
                } else {
                    j = k;
                }
            }
            int best = diff;
            best = Math.min(best, Math.abs(sorted[i] - target));
            if (i > 0) { // ← VARIATION: also check left neighbor
                best = Math.min(best, Math.abs(sorted[i - 1] - target));
            }
            maxGain = Math.max(maxGain, diff - best);
        }
        return (int) ((total - maxGain) % MOD);
    }
}
```

Time $O(n \log n)$ | **Space** $O(n)$

#1891 Cutting Ribbons

Description: Find the maximum ribbon length such that m ribbons of that length can be cut.

Template: Answer space MAXIMIZE — largest length k that still yields $\geq m$ ribbons.

Intuition: Longer pieces produce fewer ribbons, so the count flips true→false as length grows; find the largest length still giving enough ribbons. Ceiling mid avoids the infinite loop.

```

class Solution {
    public int maxLength(int[] ribbons, int k) {
        int j = 0;
        for (int ribbon : ribbons) {
            j = Math.max(j, ribbon);
        }
        int i = 1;
        int result = 0;
        while (i <= j) {
            // ← VARIATION: closed [i, j], result tracked
            int mid = i + (j - i) / 2;
            if (countRibbons(ribbons, mid) >= k) {
                result = mid;
                i = mid + 1;
            } else {
                j = mid - 1;
            }
        }
        return result;
    }
    private long countRibbons(int[] ribbons, int length) {
        long count = 0;
        for (int ribbon : ribbons) {
            count += ribbon / length;
        }
        return count;
    }
}

```

Time $O(n \log(\max(\text{ribbons})))$ | **Space** $O(1)$